



INGENIERÍA EN DESARROLLO DE SOFTWARE MULTIPLATAFORMA

Título

Entendiendo la estructura de FASTAPI y Pydantic

Profesor:

Miguel Angel Cardona Contreras

Materia:

Desarrollo para Dispositivos Inteligentes

Alumno:

Alvares Salazar Erick Sidrac

Tijuana, BC

Entendiendo la estructura de FASTAPI y Pydantic

Introducción:

Fastapi actualmente es una de las librerías más conocidas y fáciles de usar para crear aplicaciones sencillas en entornos de python ya que facilita la creación de diferentes endpoints con sus métodos específicos basados en el CRUD. Esto la ha posicionado como la opción predilecta ya sea para proyectos pequeños que no requieren de demasiados archivos o complejidad en el backend o para enseñar el funcionamiento de una API y sus endpoints ya que el uso de la herramienta Swagger muestra de forma sencilla e intuitiva la interacción con los métodos y rutas estructurados ahorrando tiempo en pruebas con aplicaciones de terceros como Postman. Por otra parte, Pydantic se complementa como una parte de fastapi ya que sirve para realizar validaciones simples del lado del servidor; básicamente indica los datos que se esperan de cada endpoint y el tipo de dato que este mismo va a aceptar, evitando así que se introduzcan elementos tipo Int en donde van strings y viceversa.

Desarrollo

Durante esta practica se realizó una serie de endpoints básicos para mostrar el funcionamiento de estas librerías con rutas pensadas por el desarrollador. Estas fueron pensadas como las más comunes de encontrar en un proyecto pequeño y que ejemplifican bien el cómo se muestra información simple de diferentes elementos.

default			^
GET	/	Read Root	▼
GET	/items/{item_id}	Read Item	▼
PUT	/items/{item_id}	Update Item	▼
GET	/users/{user_id}	Get Users	▼
PUT	/users/{user_id}	Update User	▼
GET	/products/{product_id}	Read Products	▼
PUT	/products/{product_id}	Update Product	▼

De la misma forma se evitó usar el CRUD completo con cada campo ya que solo se necesita validar el funcionamiento de las librerías.

GET

/users/{user_id}

Get Users

⌵

Parameters

Cancel

Name	Description
user_id required integer (path)	12
q string (string null) (query)	q

Execute

Clear

Responses

Curl

```
curl -X 'GET' \  
  'http://127.0.0.1:8000/users/12' \  
  -H 'accept: application/json'
```

Request URL

```
http://127.0.0.1:8000/users/12
```

PUT

/users/{user_id}

Update User

⌵

Parameters

Cancel

Name	Description
user_id required integer (path)	12

Request body

required

application/json

⌵

Edit Value

Schema

```
{  
  "name": "string",  
  "last_name": "string",  
  "age": 0  
}
```

Execute

Clear

Responses

Curl

```
curl -X 'PUT' \  
  'http://127.0.0.1:8000/users/12' \  
  -H 'accept: application/json' \  
  -H 'Content-Type: application/json' \  
  -d '{  
    "name": "string",  
    "last_name": "string",  
    "age": 0  
  }'
```

Request URL

```
http://127.0.0.1:8000/users/12
```

Server response

Code	Details
200	Response body <pre>{ "message": "usuario actualizado", "user_id": 12, "user_info": { "name": "string", "last_name": "string", "age": 0 } }</pre> Download

Response headers

Endpoints para products:

GET

/products/{product_id}

Read Products

⌵

Parameters

Cancel

Name	Description
product_id required	
integer	12
(path)	

Execute

Clear

Responses

Curl

```
curl -X 'GET' \
  'http://127.0.0.1:8000/products/12' \
  -H 'accept: application/json'
```

Request URL

http://127.0.0.1:8000/products/12

Server response

Code

Details

Execute

Clear

Responses

Curl

```
curl -X 'PUT' \
  'http://127.0.0.1:8000/products/11' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "price": 0,
    "description": "string"
  }'
```

Request URL

http://127.0.0.1:8000/products/11

Server response

Code

Details

200

Response body

```
{
  "item id": 11,
  "description": {
    "price": 0,
    "description": "string"
  }
}
```

Download

Response headers

content-length: 65

PUT

/products/{product_id}

Update Product

⌵

Parameters

Cancel

Name	Description
product_id required	
integer	11
(path)	

Request body

application/json

Edit Value

Schema

```
{
  "price": 0,
  "description": "string"
}
```

Conclusión:

El desarrollo de esta práctica permitió comprender el flujo fundamental de una API REST mediante la integración de FastAPI y Pydantic. La combinación de ambas herramientas no solo agiliza el proceso de desarrollo gracias a la validación automática de datos y la generación de documentación interactiva con Swagger, sino que también garantiza la integridad de la información que procesa el servidor. En conjunto, el ejercicio sirvió para mejorar el conocimiento de las librerías, demostrando que FastAPI y Pydantic forman una sinergia sólida, eficiente y accesible para la construcción de aplicaciones completas en python.